

MATLAB. Численное интегрирование

Вычисление определенного интеграла $q = \int_a^b f(x)dx$, $a, b \in \mathbb{R}$,
 $f : \mathbb{R} \rightarrow \mathbb{R}$ реализуется вызовом:

q=quad (fun , a , b) ;

- Здесь **a**, **b** – конечные скаляры, а **fun** – дескриптор функции, возвращающей подынтегральное выражение.
- Используется адаптивная квадратурная формула Симпсона.
- Возможен случай, когда $a, b \in \mathbb{C}$, $f : \mathbb{C} \rightarrow \mathbb{C}$. Интегрир. выполняется по отрезку компл. плоскости, соединяющему точки a и b .

q=quad (fun , a , b , tol) ;

- явное задание абсолютной погрешности **tol**

q=quad (fun , a , b , tol , trace) ;

- ненулевой параметр **trace** включает трассировку

[q, fcnt]=quad (...) ;

- возврат числа обращений к функции $f(x)$

Численное интегрирование на основе более точных адаптивных квадратурных формул Лобатто:

```
q=quadl (fun , a , b) ; q=quadl (fun , a , b , tol) ;  
q=quadl (fun , a , b , tol , trace) ; [q,fcnt]=quadl (...) ;
```

Следующие вызовы полностью аналогичны ранее рассмотренным и применимы в том случае, когда функция $y = f(x)$ возвращает матрицу или вектор:

```
Q=quadv (fun , a , b) ; Q=quadv (fun , a , b , tol) ;  
Q=quadv (fun , a , b , tol , trace) ; [Q,fcnt]=quadv (...) ;
```

След. вызовы используют высокоточные адаптивные ф-лы Гаусса:

```
q=quadgk (fun , a , b) ; [q,errbnd]=quadgk (fun , a , b , tol) ;
```

- Применимы для полубесконечных интервалов $[a, \infty)$ либо $(-\infty, b]$
- Применимы в случае, когда верхний и нижний пределы интегрирования a и b являются особыми точками
- Второй выходной параметр представляет оценку погрешности

Следующий вызов обеспечивает дополнительные возможности настройки адаптивного метода Гаусса:

```
[q,errbnd]=quadgk(fun,a,b,param1,val1,  
                  param2,val2,...) ;
```

Параметр	Действие
'AbsTol'	Абсолютная точность численного интегрирования. 10^{-10} по умолчанию
'RelTol'	Относительная точность численного интегрирования. 10^{-6} по умолчанию
'WayPoints'	Вектор из точек комплексной плоскости, определяющих путь интегрирования от точки a до точки b . Если a , b и другие точки пути интегрирования все являются действительными, то они используются в отсортированном порядке.
'MaxIntervalCount'	Макс. позволенное число разбиений отрезка интегрирования (650 по умолчанию)

Численное интегрирование функции двух переменных

Вычисление интеграла $q = \int_{x_{\min}}^{x_{\max}} dx \int_{y_{\min}}^{y_{\max}} dy f(x, y)$ в прямоугольнике

$x_{\min} \leq x \leq x_{\max}, y_{\min} \leq y \leq y_{\max}$ реализуется вызовами

q=dblquad(fun, xmin, xmax, ymin, ymax) ;

q=dblquad(fun, xmin, xmax, ymin, ymax, tol) ;

q=dblquad(fun, xmin, xmax, ymin, ymax, tol, method) ;

- Для выполнения двукратного интегрирования данная функция использует функцию **quad**, т.е. адаптивные формулы Симпсона.
- Если требуется использовать квадратурные формулы Лобатто либо другие квадратурные формулы повышенной точности, то в качестве параметра **method** используется дескриптор функции **@quad1** либо дескриптор любой другой выполняющей численное интегрирование на одномерном отрезке функции, семантика вызова которой совпадает с семантикой вызова функций **quad** и **quad1**.

Численное интегрирование $q = \int_a^b dx \int_{c(x)}^{d(x)} dy f(x, y)$ в двумерной области $a \leq x \leq b, c(x) \leq y \leq d(x)$ обеспечивается вызовами

q=quad2d(fun, a, b, c, d) ; [q, errbnd]=quad2d(...) ;

- Здесь **fun** – дескриптор функции. Входные параметры **c** и **d** также могут быть дескрипторами функций.

- Все входные функции должны быть векторизованы. Т.е.

Z=fun(X, Y) должна принимать матрицы одинакового размера и возвращать матрицу соотв. размера. **ymin=c(x)** и **ymax=d(x)** должны принимать и возвращать матрицы соотв. размера.

q=quad2d(fun, a, b, c, d, param1, val1, param2, var2, ...) ;

Параметр	Действие
'AbsTol'	Абсолютная точность числ. интегрир.
'RelTol'	Относит. точн. числ. интегрирования
'MaxFunEvals'	Макс. допустимое число обращений к ф-ии fun .
'Singular'	true , если $f(x)$ сингулярна вблизи границ

Численное интегрирование функции трех переменных

Трехмерное интегрирование $q = \int_{x_{\min}}^{x_{\max}} dx \int_{y_{\min}}^{y_{\max}} dy \int_{z_{\min}}^{z_{\max}} dz f(x, y, z)$ в параллелепипеде $x_{\min} \leq x \leq x_{\max}$, $y_{\min} \leq y \leq y_{\max}$, $z_{\min} \leq z \leq z_{\max}$ выполняется посредством вызовов, аналогичных вызовам функции **dblquad**

```
q=triplequad(fun,xmin,xmax,ymin,ymax,zmin,zmax);  
q=triplequad(fun,xmin,xmax,ymin,ymax,zmin,zmax,  
tol);  
q=triplequad(fun,xmin,xmax,ymin,ymax,zmin,zmax,  
tol,method);
```

МАТЛАВ. Дискретное преобразование Фурье.

Ряд Фурье T -периодической функции

$$x(t) = \sum_{k=-\infty}^{\infty} c_k e^{2ik\pi t/T}, \quad c_k = \frac{1}{T} \int_0^T x(t) e^{-2ik\pi t/T} dt \quad (1)$$
$$t \in \mathbb{R}, \quad x: \mathbb{R} \rightarrow \mathbb{C}, \quad i = \sqrt{-1}$$

Прямое и обратное преобразование Фурье ($T \rightarrow \infty$)

$$\hat{x}(\omega) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} x(t) e^{-i\omega t} dt, \quad x(t) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} \hat{x}(\omega) e^{i\omega t} d\omega \quad (2)$$

Дискретное преобразование Фурье (ДПФ)

$$\mathbf{x} = (x_0, x_1, \dots, x_{N-1})^T \in \mathbb{C}^N, \quad \mathbf{y} = (y_0, y_1, \dots, y_{N-1})^T \in \mathbb{C}^N$$
$$y_k = \sum_{j=0}^{N-1} x_j \omega_N^{-kj}, \quad x_k = \frac{1}{N} \sum_{j=0}^{N-1} y_j \omega_N^{kj}, \quad k = 0, 1, \dots, N-1; \quad \omega_N = e^{2\pi i/N} \quad (3)$$

- Соотв. индексов: y_0 и c_0 , y_1 и c_1 , y_2 и $c_2, \dots$, y_{N-1} и c_{-1} , y_{N-2} и c_{-2}
- Быстрое преобразование Фурье (БПФ): $N = 2^M$, число действий:
 $\sim N \log_2 N$

$$\omega_{nm}^m = e^{2\pi i m / (nm)} = e^{2\pi i / n} = \omega_n \quad (4)$$

$$N = N_1 N_2; \quad y_k = \sum_{j=0}^{N-1} x_j \omega_N^{-kj}, \quad k = 0, 1, \dots, N-1; \quad \omega_N = e^{2\pi i / N} \quad (5)$$

индексы k и j делаем двумерными, но проходим в разных направлениях

$$\begin{aligned} j &= N_2 j_1 + j_2, \quad j_1 = 0, \underline{N_1 - 1}, \quad j_2 = 0, \underline{N_2 - 1} \\ k &= N_1 k_2 + k_1, \quad k_2 = 0, N_2 - 1, \quad k_1 = 0, N_1 - 1 \end{aligned} \quad (6)$$

$$\begin{aligned} y_{N_1 k_2 + k_1} &= \sum_{j_1=0}^{N_1-1} \sum_{j_2=0}^{N_2-1} x_{N_2 j_1 + j_2} \omega_N^{-(N_1 k_2 + k_1)(N_2 j_1 + j_2)} = \\ &= \sum_{j_1=0}^{N_1-1} \sum_{j_2=0}^{N_2-1} x_{N_2 j_1 + j_2} \omega_N^{-(\textcolor{violet}{N}_1 \textcolor{violet}{N}_2 k_2 j_1 + N_1 k_2 j_2 + N_2 k_1 j_1 + k_1 j_2)} = \\ &= \sum_{j_2=0}^{N_2-1} \omega_N^{-(N_1 k_2 + k_1) j_2} \sum_{j_1=0}^{N_1-1} x_{N_2 j_1 + j_2} \omega_N^{-N_2 k_1 j_1} = \\ &= \sum_{j_2=0}^{N_2-1} \omega_N^{-(N_1 k_2 + k_1) j_2} \sum_{j_1=0}^{\textcolor{blue}{N}_1-1} \textcolor{blue}{x}_{N_2 j_1 + j_2} \textcolor{blue}{\omega}_{\textcolor{blue}{N}_1}^{-k_1 j_1}, \quad k_2 = 0, N_2 - 1, k_1 = 0, N_1 - 1 \end{aligned}$$

– $\textcolor{blue}{\text{не зависят}}$ от k_2 ; $N = N_1 N_2 : \quad \underline{\underline{O}}(N^2) \rightarrow \underline{\underline{O}}(N(N_1 + N_2)) \quad (7)$

$$\begin{aligned} N &= N_1 N_2 \dots N_p : \quad \underline{\underline{O}}(N^2) \rightarrow \underline{\underline{O}}(N(N_1 + N_2 + \dots + N_p)) \\ N &= 2^p : \quad \underline{\underline{O}}(N^2) \rightarrow \underline{\underline{O}}(N \log_2 N) \end{aligned} \quad (8)$$

- MATLAB, SciLAB и Octave используют либо свободно распространяемые библиотеки ДПФ FFTW3, либо аналогичные интерфейсы коммерческой библиотеки Intel MKL

$Y = \text{fft}(X) ;$

- дискретное преобразование Фурье вектора X . Если X – матрица, то ДПФ выполняется для каждого ее столбца. Если X – мультиразмерный массив, то ДПФ выполняется по измерению, которому соответствует первый индекс, диапазон изменения которого превышает 1.

$Y = \text{fft}(X, n) ;$

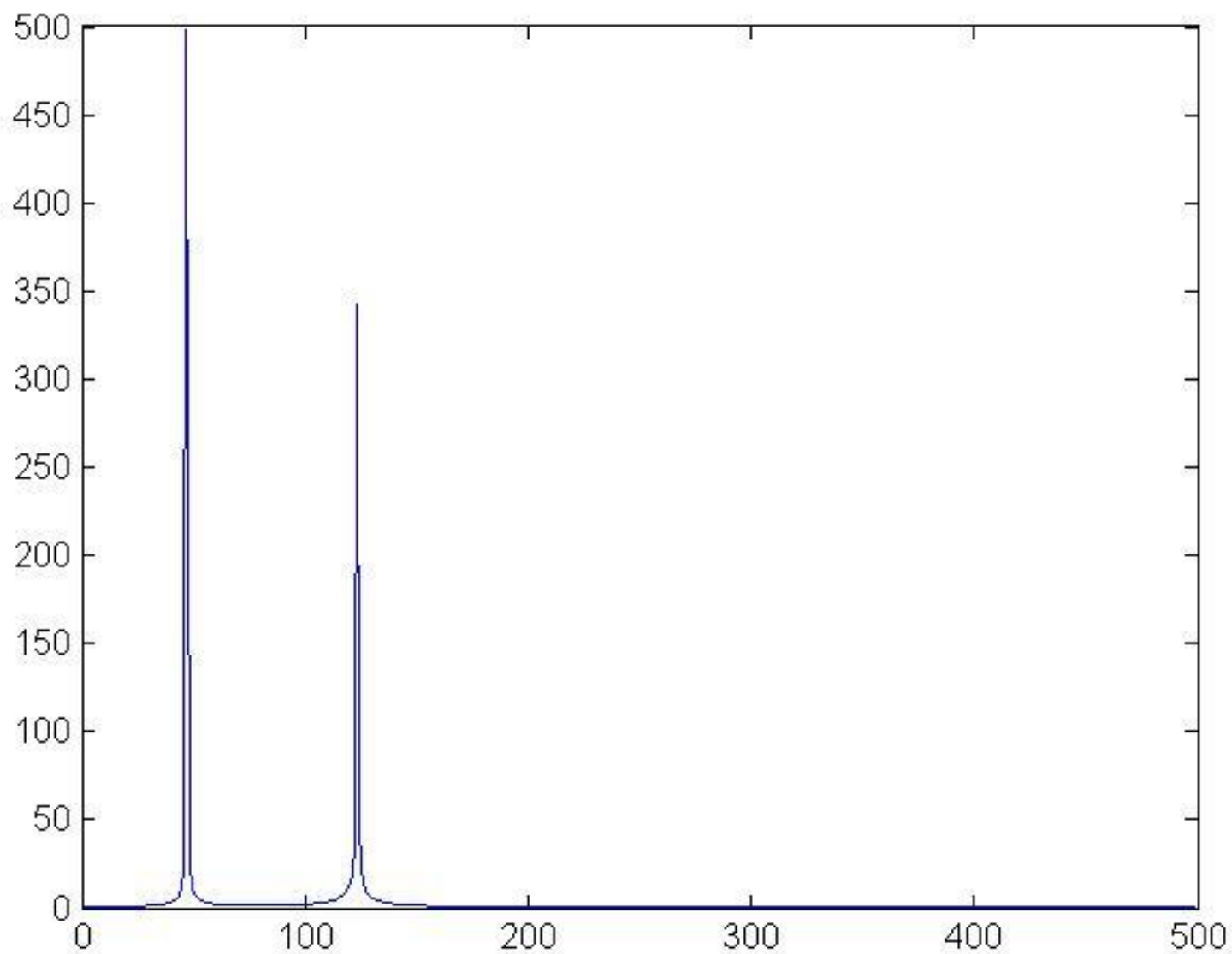
- n -точечное ДПФ. Если длина вектора X превосходит n , то излишние элементы не участвуют в преобразовании Фурье. Если длина вектора X меньше, чем n , то недостающие элементы заменяются нулем.

$Y = \text{fft}(X, [], \text{dim}) ; \quad Y = \text{fft}(X, n, \text{dim}) ;$

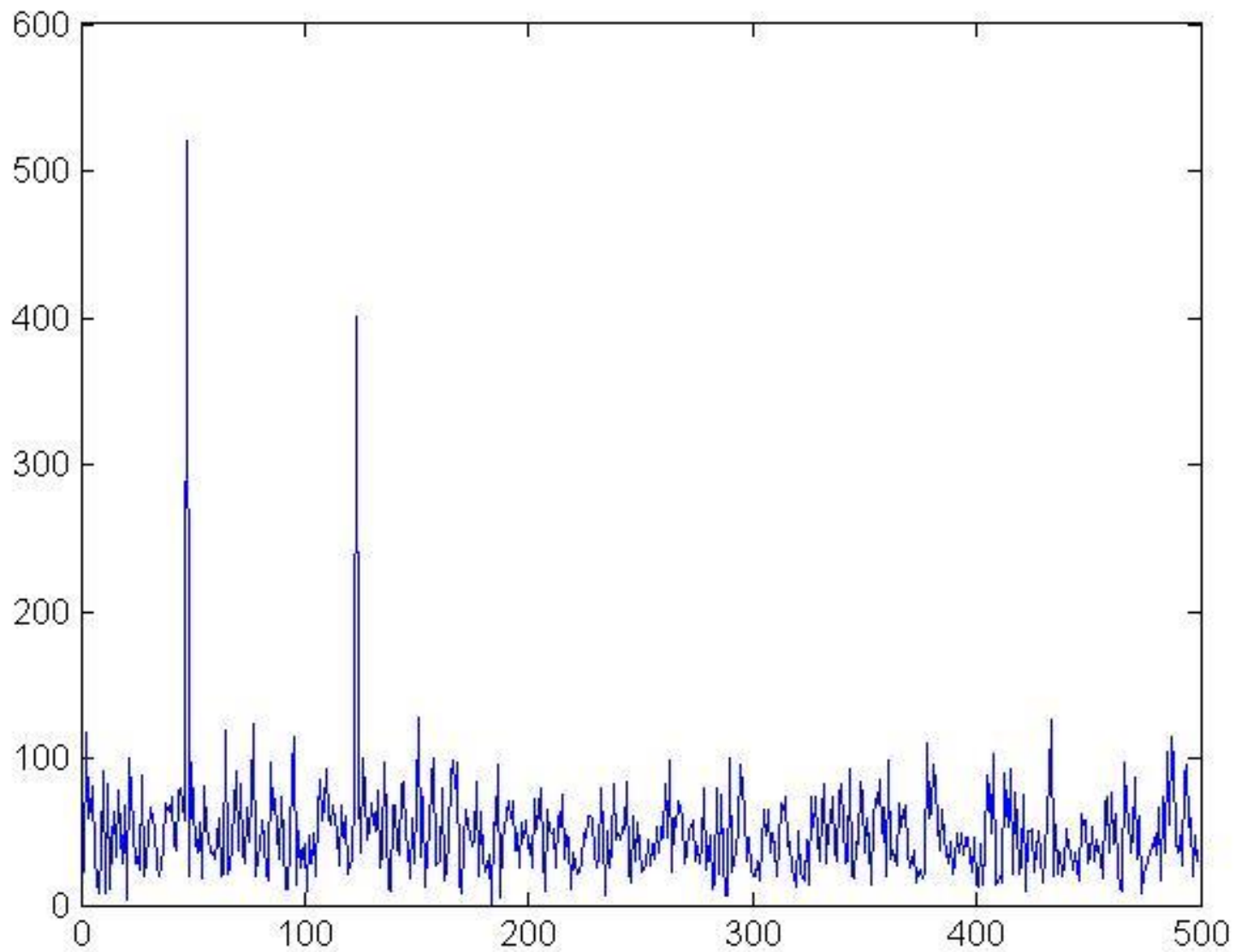
- выполняется ДПФ по измерению, которое определяется индексом s номером n .

Пример. Анализ зашумленного сигнала

```
%Анализ частотного спектра зашумленного сигнала
W=1000;          % Характерная частота в Гц
dt=1/W;          % Характерный период между временными от-
счетами
N=500;           % 2N+1 - Число точек в сигнале
dw=W/(2*N);      % Характерный шаг частоты
t=(0:2*N)*dt;    %Набор отсчетов времени
w1=47; w2=123;   %Частоты исходного сигнала в Гц
A1=1; A2=0.7;    %Амплитуды исходного сигнала
x=A1*sin(2*pi*w1*t)+A2*cos(2*pi*w2*t); %Исходный сигнал
y=x+(A1+A2)*randn(size(t)); %Зашумленный сигнал
w=dw*(0:N-1);    %Вектор из отсчетов частот
X=fft(x); %Преобразование Фурье исходного сигнала
figure; plot(w,abs(X(1:N)));
Y=fft(y); %Преобразование Фурье зашумленного сигнала
figure; plot(w,abs(Y(1:N)));
```



Спектр исходного сигнала



Спектр зашумленного сигнала

Обратное ДПФ реализуется вызовами:

Y=ifft(X) ; Y=ifft(X,n) ;

Y=ifft(X, [], dim) ; Y=ifft(X,n,dim) ;

Если $\mathbf{x} = (x_0, x_1, \dots, x_{N-1})^T \in \mathbb{R}^N$, то $y_{N-k} = \bar{y}_k$, $k = 1, 2, 3, \dots$. Для того, чтобы явно это учесть:

Y=ifft(..., 'symmetric') ;

«Центрирование» спектра. Преобразование

YY=fftshift(Y) ;

Переупорядочивает элементы массива Y так, что центральному элементу массива Y соответствует нулевое значение частоты. Обратное преобразование реализуется вызовом

Y=ifftshift(YY) ;

Двойные ряды Фурье

$$x(t_1, t_2) = \sum_{k_1=-\infty}^{\infty} \sum_{k_2=-\infty}^{\infty} c_{k_1 k_2} e^{2\pi i \left(\frac{k_1 t_1}{T_1} + \frac{k_2 t_2}{T_2} \right)}, \quad c_{k_1 k_2} = \frac{1}{T_1 T_2} \int_0^{T_1} \int_0^{T_2} x(t_1, t_2) e^{-2\pi i \left(\frac{k_1 t_1}{T_1} + \frac{k_2 t_2}{T_2} \right)} dt_1 dt_2$$

Двукратное преобразование Фурье ($T_1 \rightarrow \infty, T_2 \rightarrow \infty$)

$$\hat{x}(\omega_1, \omega_2) = \frac{1}{2\pi} \int_{-\infty}^{\infty} x(t_1, t_2) e^{-i(\omega_1 t_1 + \omega_2 t_2)} dt_1 dt_2$$
$$x(t_1, t_2) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{x}(\omega_1, \omega_2) e^{i(\omega_1 t_1 + \omega_2 t_2)} d\omega_1 d\omega_2$$

Двукратное дискретное преобразование Фурье

$$X = \{x_{k_1 k_2}\} \in \mathbb{C}^{N_1 \times N_2}, \quad Y = \{y_{k_1 k_2}\} \in \mathbb{C}^{N_1 \times N_2}$$

$$y_{k_1 k_2} = \sum_{j_1=0}^{N_1-1} \sum_{j_2=0}^{N_2-1} x_{j_1 j_2} \omega_{N_1}^{-k_1 j_1} \omega_{N_2}^{-k_2 j_2}, \quad x_{k_1 k_2} = \frac{1}{N_1 N_2} \sum_{j_1=0}^{N_1-1} \sum_{j_2=0}^{N_2-1} y_{j_1 j_2} \omega_{N_1}^{k_1 j_1} \omega_{N_2}^{k_2 j_2}$$

$$\omega_{N_1} = e^{2\pi i / N_1}, \quad \omega_{N_2} = e^{2\pi i / N_2}, \quad i = \sqrt{-1}$$

$$k_1 = 0, 1, \dots, N_1 - 1; \quad k_2 = 0, 1, \dots, N_2 - 1$$

Двукратное ДПФ реализуется вызовом

$Y = \text{fft2}(X) ;$
 $Y = \text{fft2}(X, m, n) ;$

- выполняет двукратное ДПФ для матрицы X размерности $m \times n$.
Если размеры матрицы X меньше, чем $m \times n$, то она дополняется нулями.

Обратное двумерное ДПФ реализуется вызовами:

$Y = \text{fft2}(X) ; Y = \text{fft2}(X, m, n) ; Y = \text{fft2}(..., 'symmetric') ;$

- последний вызов целесообразен, если исходная матрица представляет собой ДПФ матрицы с вещественными элементами.

$YY = \text{fftshift}(Y) ; YY = \text{fftshift}(Y, \text{dim}) ;$

- сопоставляет нулевую частоту центральному элементу массива YY

$Y = \text{ifftshift}(YY) ; Y = \text{ifftshift}(YY, \text{dim}) ;$

- реализуют обратное переупорядочивание

n-кратное дискретное преобразование Фурье

$$X = \{x_{k_1 k_2 \dots k_n}\} \in \mathbb{C}^{N_1 \times N_2 \times \dots \times N_n}, \quad Y = \{y_{k_1 k_2 \dots k_n}\} \in \mathbb{C}^{N_1 \times N_2 \times \dots \times N_n}$$

$$y_{k_1 k_2 \dots k_n} = \sum_{j_1=0}^{N_1-1} \sum_{j_2=0}^{N_2-1} \dots \sum_{j_n=0}^{N_n-1} x_{j_1 j_2 \dots j_n} \omega_{N_1}^{-k_1 j_1} \omega_{N_2}^{-k_2 j_2} \dots \omega_{N_n}^{-k_n j_n}$$

$$x_{k_1 k_2 \dots k_n} = \frac{1}{N_1 N_2 \dots N_n} \sum_{j_1=0}^{N_1-1} \sum_{j_2=0}^{N_2-1} \dots \sum_{j_n=0}^{N_n-1} y_{j_1 j_2 \dots j_n} \omega_{N_1}^{k_1 j_1} \omega_{N_2}^{k_2 j_2} \dots \omega_{N_n}^{k_n j_n}$$

$$\omega_{N_1} = e^{2\pi i / N_1}, \quad \omega_{N_2} = e^{2\pi i / N_2}, \dots, \quad \omega_{N_n} = e^{2\pi i / N_n}, \quad i = \sqrt{-1}$$

$$k_1 = 0, 1, \dots, N_1 - 1; \quad k_2 = 0, 1, \dots, N_2 - 1; \dots; \quad k_n = 0, 1, \dots, N_n - 1$$

$$\mathbf{Y} = \mathbf{fft}(\mathbf{X});$$

- для **n**-мерного массива реализует n-кратное прямое ДПФ. Выходной массив **Y** имеет ту же размерность, что и входной.

$$\mathbf{Y} = \mathbf{fft}(\mathbf{X}, \mathbf{siz});$$

- реализует **n**-кратное прямое ДПФ для фрагмента массива **X**, размерность которого задается компонентами вектора **siz**. Если массив меньше, чем размерности, указанные в **siz**, он дополняется нулями.

Обратное **n**-кратное ДПФ реализуется вызовами:

```
Y=ifftn(X) ; Y=ifftn(X,siz) ;  
Y=ifftn(X,'symmetric') ;
```

Если массив **Y** — результат **n**-кратного прямого ДПФ, то сопоставлению его центральному элементу нулевых частот реализуется вызовами

```
YY=fftshift(Y) ; YY=fftshift(Y,dim) ;
```

а обратное переупорядочивание реализуется вызовами

```
Y=ifftshift(YY) ; Y=ifftshift(YY,dim) ;
```